Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay

Laboratory Reportl in IT 112: Web Systems and Technologies

2nd Semester A.Y 2025 - 2026

NEXUSWrites

"Your Tech, Your Story."

Group Name:

Powerpuffgals

Group Members:

Kris Ann B. Apoon

April Grace R. Dacanay

Ivy Angel R. Hidalgo

Princes Mae L. Mateo

Milan R. Rellora

Course & Block:

BSIT-2A

I. Introduction

For this laboratory activity, our group worked on Phase 2- Backend Development and API Integration for our final project in IT 112: Web systems and Technologies. In this phase we focused on building the backend part of the system using Node.js, Express.js, MongoDB, and Mongoose. The backend is important because it handles data, processes requests, and connects the frontend to the database. During this activity, we created the backend folder structure, installed the required packages, connected our project to MongoDB, created Mongoose models, and built API routes for CRUD operations. We also tested the API using Postman and Thunder Client and committed our progress to GitHub. This phase helped us understand how backend development works and how API's are created and tested.

II. Objectives

At the end of this activity, we were able to:

1. Set up a working backend server using Node.js and Express.js.
2. Connect the backend to MongoDB using Mongoose.
3. Create REST API routes for CRUD operations.
4. Test backed endpoints using Postman/Thunder Client.
5. Organize backend files into a modular folder structure.

III. Overview of Backend Setup

For this phase, we created a /backend folder inside our GitHub repository. Inside it, we added the main files and folders needed for backend development.

Backend Folder Structure

Bash

```
/backend
├── config
├── /controllers
├── /middleware
├── /models
├── /routes
├── /server
```

This folder structure helped us organise our backend files properly. The server.js file contains the main server setup, the routes folder contains API routes, the models folder contains the database models, the config folder contains the MongoDB connection file, and the .env file stores the environment variables.

IV. Code Snippets

1. Server.js

```
JS server.js ×
backend > JS server.js > ...
1  require('dotenv').config();
2  const express = require('express');
3  const cors = require('cors');
4  const dns = require('dns');
5  const connectDB = require('./config/db');
6  const { errorHandler } = require('./middleware/errorMiddleware');
7
8  // Fix for network/DNS environments in Bicol (Preserved)
9  dns.setServers(['8.8.8.8', '8.8.4.4']);
10
11 const app = express();
12
13 // Connect to Database
14 connectDB();
15
16 // Global Middleware (Updated with limits to fix "Entity Too Large")
17 app.use(cors());
18 app.use(express.json({ limit: '10mb' }));
19 app.use(express.urlencoded({ limit: '10mb', extended: true }));
20
21 // Logging Middleware (Preserved)
22 app.use((req, res, next) => {
23     console.log(`🚀 ${req.method} request to: ${req.url}`);
24     next();
25 });
26
27 // Routes (Preserved)
28 app.use('/api/users', require('./routes/userRoutes'));
29 app.use('/api/posts', require('./routes/postRoutes'));
30 app.use('/api/github', require('./routes/githubRoutes'));
31 app.use('/api/comments', require('./routes/commentRoutes'));
32 app.use('/api/notifications', require('./routes/notificationRoutes'));
33
34 // Root Route
35 app.get('/', (req, res) => {
```

2. Sample Model

- **Category.js**

```
JS Category.js ×
backend > models > JS Category.js > ...
1  const mongoose = require('mongoose');
2
3  const CategorySchema = new mongoose.Schema({
4      name: { type: String, required: true, unique: true },
5      description: String,
6      post_count: { type: Number, default: 0 },
7      created_at: { type: Date, default: Date.now }
8  });
9
10 module.exports = mongoose.model('Category', CategorySchema);
```

- **Comment.js**

```
JS Comment.js ×
backend > models > JS Comment.js > ...
const mongoose = require('mongoose');

const commentSchema = new mongoose.Schema({
    // Link to the Post being commented on
    postId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Post',
        required: true
    },
    // The name of the person (for quick display)
    author: {
        type: String,
        required: true
    },
    // Link to the User's account (Crucial for profile linking!)
    authorId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    text: {
        type: String,
        required: true
    },
    // For Replies: Points back to the original comment
    parentCommentId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Comment',
        default: null
    },
    // Array of IDs pointing to all replies for this specific comment
    replies: [{
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Comment'
    }]
```

- **Notification.js**

```
JS Notification.js X
backend > models > JS Notification.js > ...
const mongoose = require('mongoose');

const notificationSchema = new mongoose.Schema({
    recipient: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    sender: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    type: {
        type: String,
        enum: ['like', 'follow', 'comment'],
        required: true
    },
    post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post' },
    isRead: { type: Boolean, default: false },
}, { timestamps: true });

module.exports = mongoose.model('Notification', notificationSchema);
```

- **Post.js**

```
JS Post.js ×
backend > models > JS Post.js > ...
const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
    title: { type: String, required: true },
    content: { type: String, required: true },
    // UPDATE: Use ObjectId to link to the User model
    author: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    image: { type: String, default: null },
    tags: { type: [String], default: [] },
    category: { type: String, default: "General" },
    likes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }]
}, { timestamps: true });

module.exports = mongoose.model('Post', postSchema);
```

- **User.js**

```
JS User.js ×
backend > models > JS User.js > ...
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    dob: { type: Date, required: true },
    role: { type: String, default: 'student' },
    bio: { type: String, default: "" },
    avatar: { type: String, default: null },

    // --- INTERESTS DROPDOWN ---
    interests: {
        type: [String],
        enum: ['web development', 'ui/ux design', 'database management', 'mobile dev', 'cybersecurity', 'systems ecology',
        default: []
    },

    // --- ERD FIELDS ---
    githubUsername: { type: String, default: "" },
    age: { type: Number },
    gender: { type: String, enum: ['male', 'female', 'other', 'prefer not to say'], default: 'other' },

    // --- SETTINGS DROPDOWNS ---
    settings: {
        theme: { type: String, enum: ['light', 'dark'], default: 'light' },
        fontSize: { type: String, enum: ['small', 'medium', 'large'], default: 'medium' },
        language: {
            type: String,
            enum: ['english', 'tagalog', 'bicolano', 'spanish'],
            default: 'english'
        },
        timezone: { type: String, default: 'UTC+8' }
    },
```

3. Routes

- **commentRoutes.js**

```
JS commentRoutes.js ×
backend > routes > JS commentRoutes.js > ...
const express = require('express');
const router = express.Router();
const commentController = require('../controllers/commentController');
const { protect } = require('../middleware/authMiddleware');

// POST a comment (Protected)
router.post('/', protect, commentController.createComment);

// GET comments for a post (Public)
router.get('/:postId', commentController.getCommentsByPost);

module.exports = router;
```

- **githubRoutes.js**

```
JS githubRoutes.js ×
backend > routes > JS githubRoutes.js > ...
const express = require('express');
const router = express.Router();
const githubController = require('../controllers/githubController');

// URL: /api/github/:username
router.get('/:username', githubController.getUserRepos);

module.exports = router;
```

- **notificationRoutes.js**

```
JS notificationRoutes.js ×
backend > routes > JS notificationRoutes.js > ...
const express = require('express');
const router = express.Router();
const { getNotifications, markAsRead } = require('../controllers/notificationController');
const { protect } = require('../middleware/authMiddleware');

router.get('/', protect, getNotifications);
router.put('/read', protect, markAsRead);

module.exports = router;
```

- **postRoutes.js**

```
JS postRoutes.js ×
backend > routes > JS postRoutes.js > ...
const express = require('express');
const router = express.Router();
const postController = require('../controllers/postController');
const upload = require('../config/cloudinary');
const { protect } = require('../middleware/authMiddleware');

// --- 1. POST OPERATIONS ---
router.post('/', protect, upload.single('image'), postController.createPost);
router.get('/', postController.getAllPosts);

// --- 2. DISCOVERY ---
router.get('/search', postController.searchPosts);
router.get('/following-feed', protect, postController.getFollowingFeed);

// --- 3. ENGAGEMENT ---
router.put('/like/:id', protect, postController.likePost);

// --- 4. EDIT & DELETE (Ownership Protected) ---
router.put('/:id', protect, upload.single('image'), postController.updatePost);
router.delete('/:id', protect, postController.deletePost);

// --- 5. DATA FETCHING ---
router.get('/:id', postController.getPostById);

module.exports = router;
```

- **userRoutes.js**

```
JS userRoutes.js ×
backend > routes > JS userRoutes.js > router.get('/') callback
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');
const upload = require('../config/cloudinary');
const { protect } = require('../middleware/authMiddleware');
const User = require('../models/User');

// 1. AUTH
router.post('/register', upload.single('avatar'), userController.register);
router.post('/login', userController.login);

// 2. SEARCH & STATS
router.get('/search', userController.searchUsers);
router.get('/stats', protect, userController.getUserStats);
router.get('/friends', protect, userController.getFriends);
router.get('/me', protect, (req, res) => res.json(req.user));

// 3. PROFILE UPDATES (Use this URL: /api/users/update)
router.put('/update', protect, upload.single('avatar'), userController.updateUserProfile);
router.put('/follow/:id', protect, userController.toggleFollow);

// 4. DYNAMIC PROFILE
router.get('/profile/:id', userController.getUserProfile);

// 5. SYSTEM LIST (Get all users)
router.get('/', async (req, res) => {
    try {
        const users = await User.find().select('-password');
        res.json(users);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

module.exports = router;
```

V. API TESTING

1. **Users**

Testing: The team successfully performed full CRUD (Create, Read, Update, Delete) testing.

POST /api/users: Successfully created and added new users to the MongoDB database.

GET /api/users: Verified that all saved user data could be retrieved and displayed.

PUT /api/users/:id: Confirmed that specific user information could be updated.

DELETE /api/users/:id: Successfully removed selected users from the system.

Implementation: Routes were also created for user authentication (login and registration), searching users, and retrieving user-specific statistics.

2. **Follow and Followed**

Implementation: A specific route was established to handle social interactions between users.

Testing: The PUT /api/users/follow/:id route was tested, returning a "Followed successfully" message upon a successful request.

3. **Posts and Likes**

Implementation: The system uses a dedicated Post.js model to store titles, content, and images.

Testing:

Engagement: Tested the PUT /api/posts/like/:id route to allow users to interact with content.

Discovery: Verified routes for searching posts and fetching a "following-feed" for a personalized user experience.

Management: Tested standard operations including creating, updating, and deleting posts (with ownership protection).

4. **Comments**

Implementation: A Comment.js model was developed to link comments to specific posts and their authors.

Testing: POST /api/comments: Successfully tested adding new comments to a post.

GET /api/comments/:postId: Verified the ability to retrieve all comments associated with a specific post. Notifications Implementation: A Notification.js model was created to track interactions like 'like', 'follow', and 'comment'.

Testing: GET /api/notifications: Tested to ensure users can fetch their list of alerts.

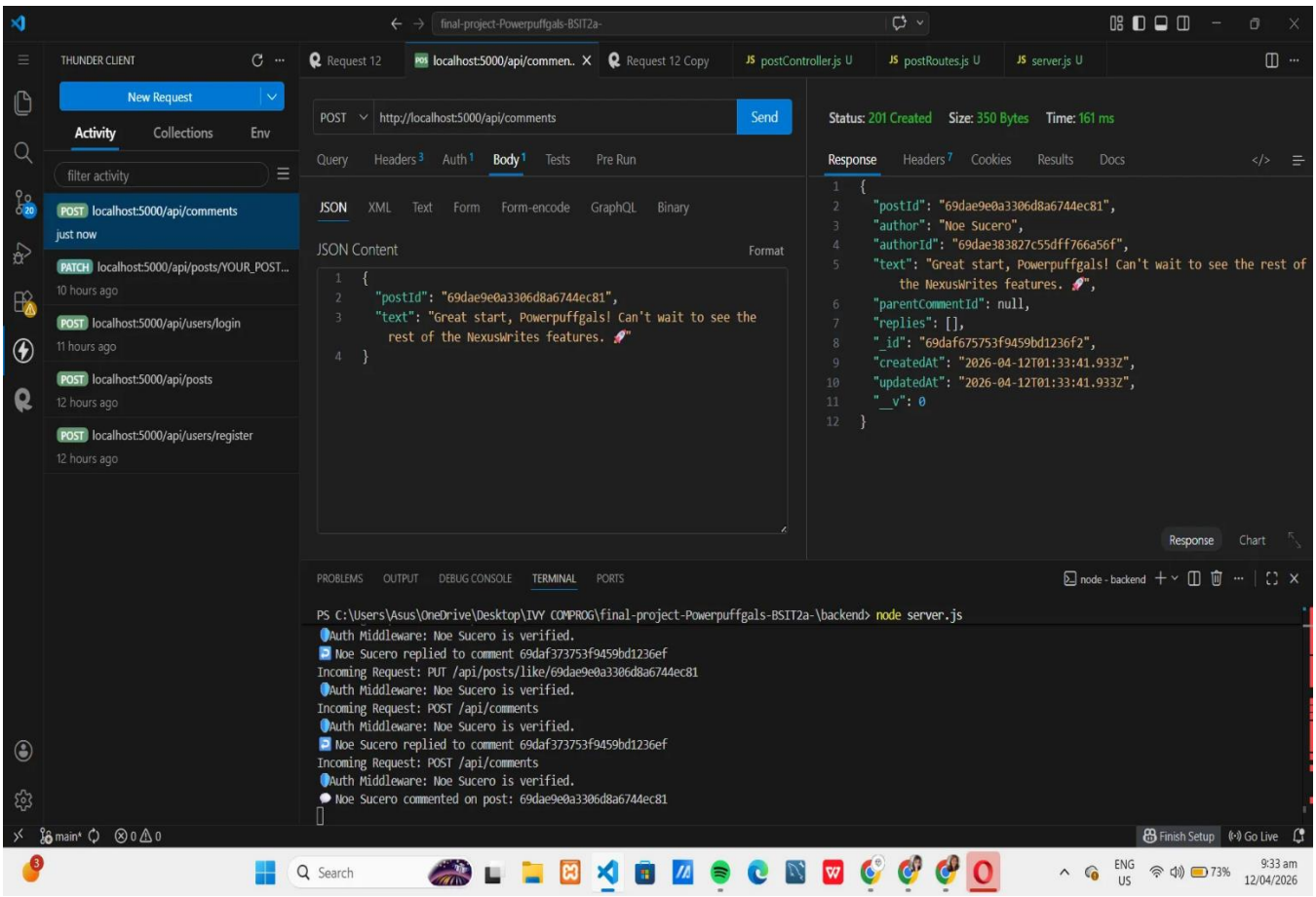
PUT /api/notifications/read: Verified that users can mark their notifications as read.

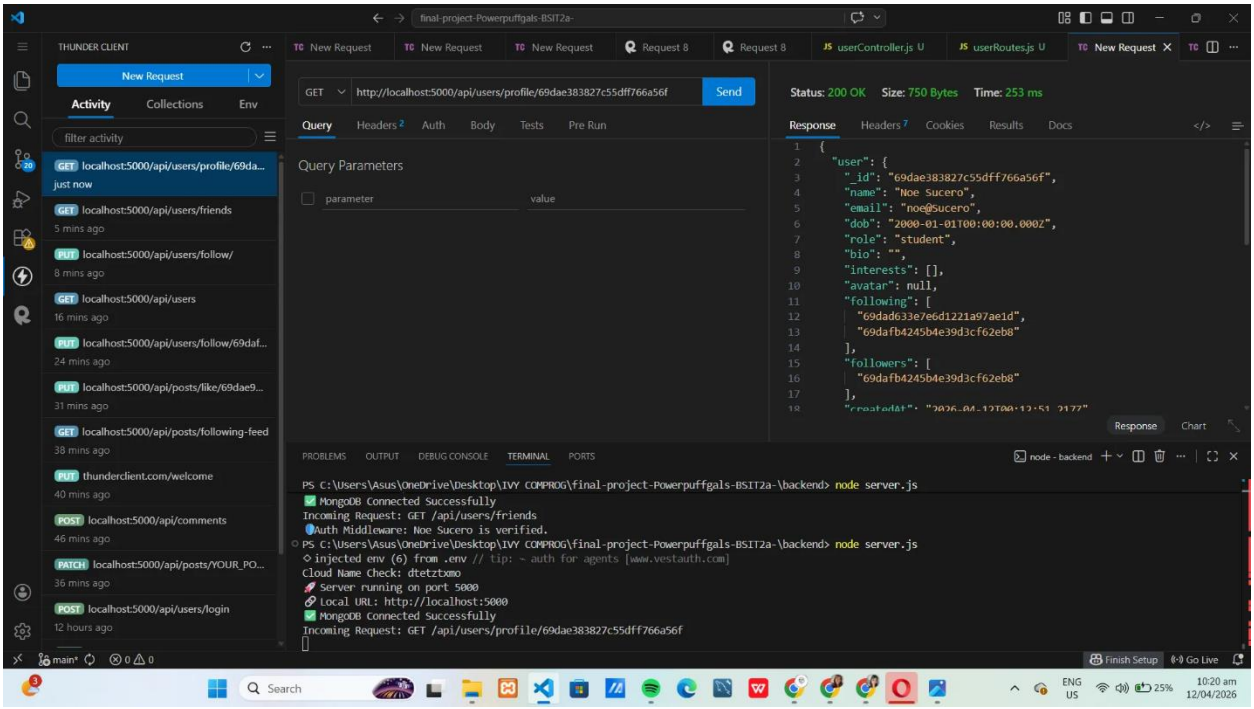
5. **Notifications**

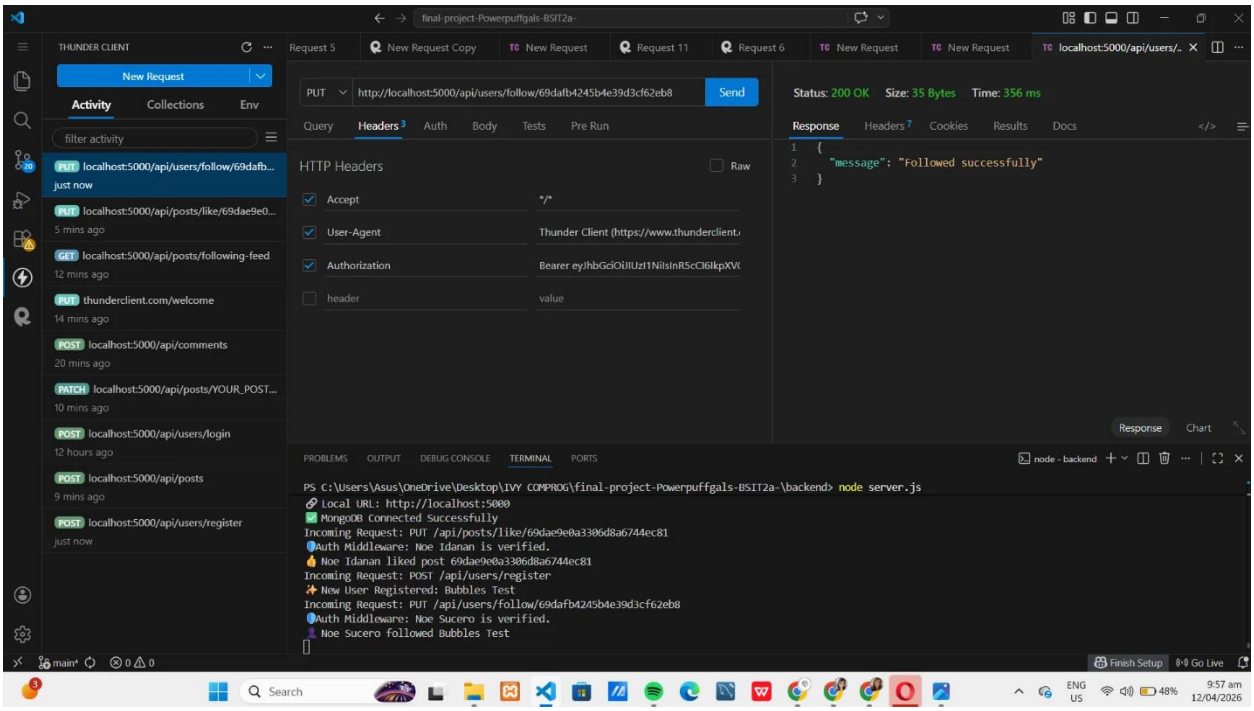
Implementation: A Notification.js model was created to track interactions like 'like', 'follow', and 'comment'.

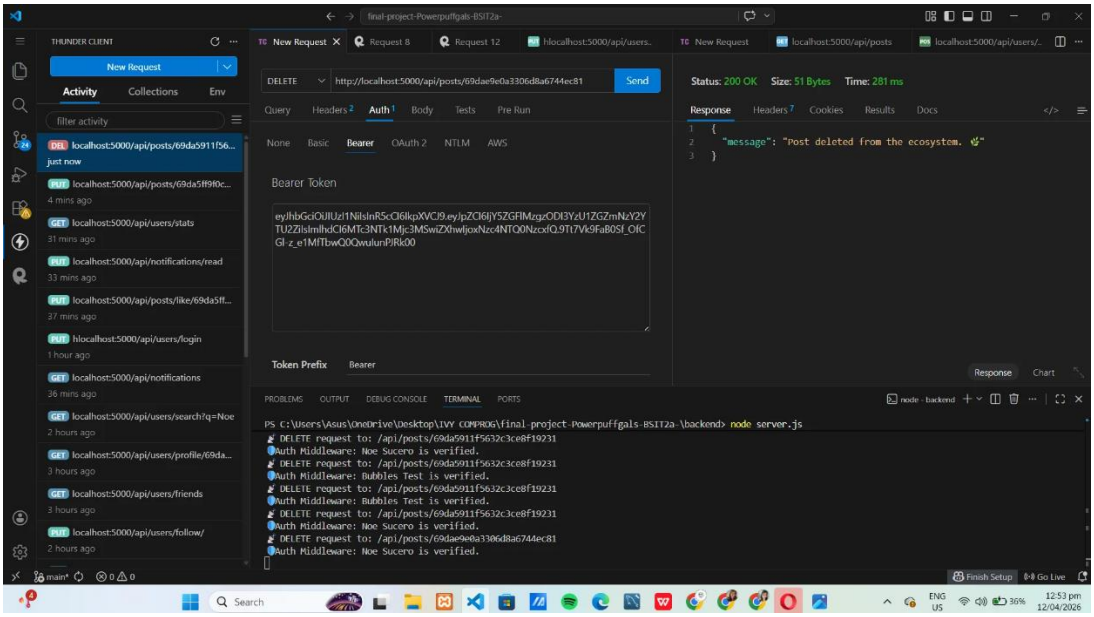
Testing: GET /api/notifications: Tested to ensure users can fetch their list of alerts. PUT /api/notifications/read: Verified that users can mark their notifications as read.

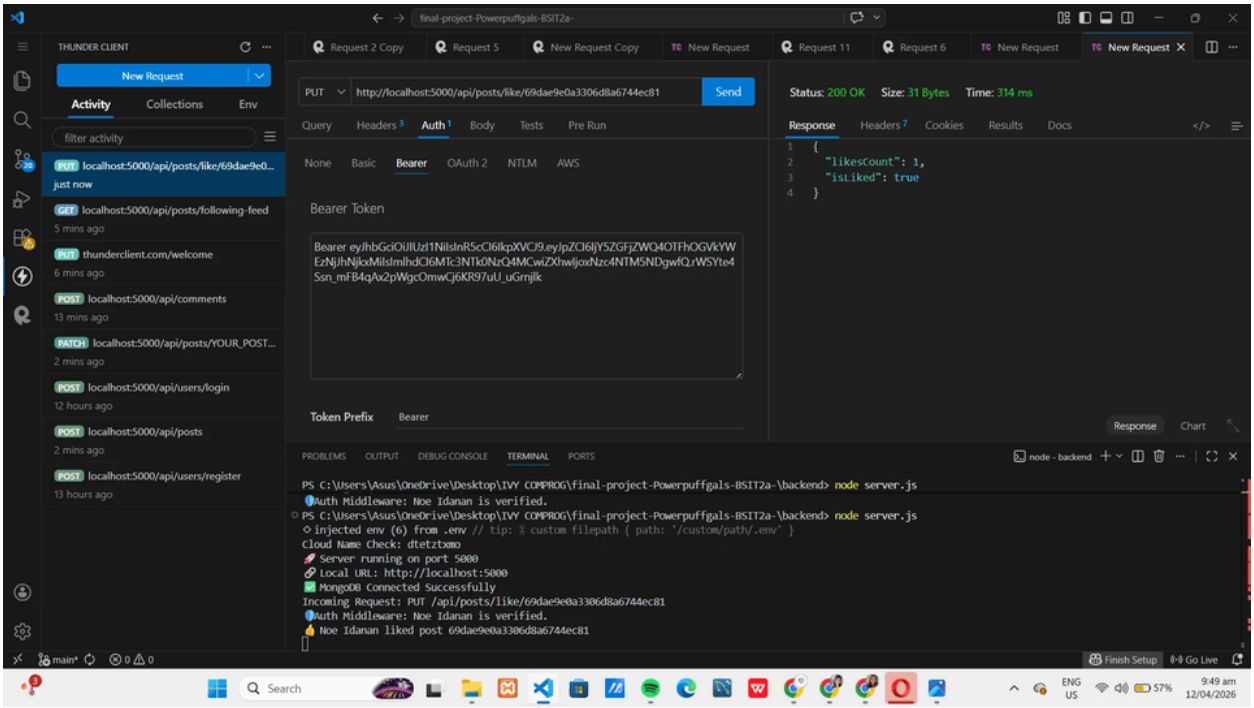
Sample Control API Testing

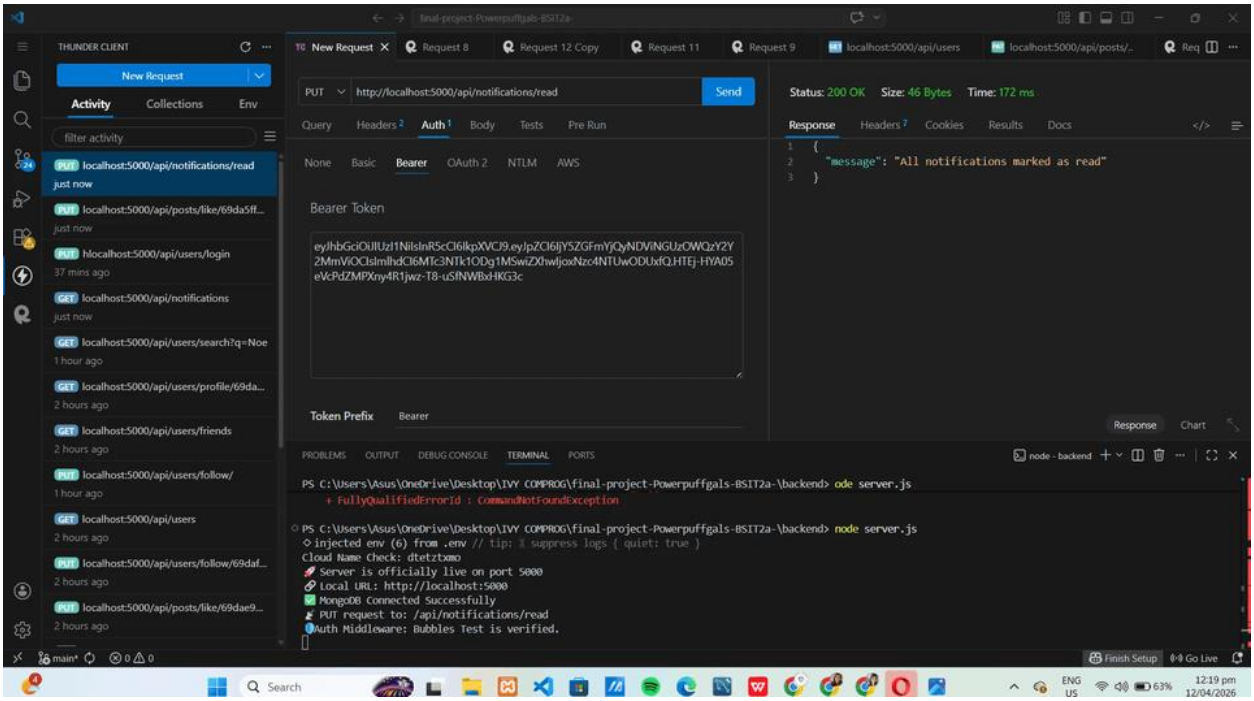



THUNDER CLIENT
New Request
Activity
Collections
Env
filter activity
GET localhost:5000/api/users/profile/69da... just now
GET localhost:5000/api/users/friends 5 mins ago
PUT localhost:5000/api/users/follow/ 8 mins ago
GET localhost:5000/api/users 16 mins ago
PUT localhost:5000/api/users/follow/69daf... 24 mins ago
PUT localhost:5000/api/posts/like/69dae9... 31 mins ago
GET localhost:5000/api/posts/following-feed 38 mins ago
PUT thunderclient.com/welcome 40 mins ago
POST localhost:5000/api/comments 46 mins ago
PATCH localhost:5000/api/posts/YOUR_PO... 36 mins ago
POST localhost:5000/api/users/login 12 hours ago
GET http://localhost:5000/api/users/profile/69dae383827c55dff766a56f
Send
Query
Headers 2
Auth
Body
Tests
Pre Run
Query Parameters
parameter
value
Status: 200 OK
Size: 750 Bytes
Time: 253 ms
Response
Headers 7
Cookies
Results
Docs
{
"user": {
"_id": "69dae383827c55dff766a56f",
"name": "Noe Sucero",
"email": "noe@sucero",
"dob": "2000-01-01T00:00:00.000Z",
"role": "student",
"bio": "",
"interests": [],
"avatar": null,
"following": [
"69dad633e7e6d1221a97ae1d",
"69dafb4245b4e39d3cf62eb8"
],
"followers": [
"69dafb4245b4e39d3cf62eb8"
],
Response
Chart
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
node - backend
PS C:\Users\Asus\OneDrive\Desktop\IVY COMPROG\final-project-Powerpuffgals-BSIT2a-\backend> node server.js
MongoDB Connected Successfully
Incoming Request: GET /api/users/friends
Auth Middleware: Noe Sucero is verified.
PS C:\Users\Asus\OneDrive\Desktop\IVY COMPROG\final-project-Powerpuffgals-BSIT2a-\backend> node server.js
injected env (6) from .env // tip: ~ auth for agents [www.vestauth.com]
Cloud Name Check: dtetztxmo
Server running on port 5000
Local URL: http://localhost:5000
MongoDB Connected Successfully
Incoming Request: GET /api/users/profile/69dae383827c55dff766a56f
Finish Setup
Go Live
Search
ENG US
25%
10:20 am
12/04/2026


THUNDER CLIENT
New Request
Activity
Collections
Env
filter activity
PUT localhost:5000/api/users/follow/69dafb... just now
PUT localhost:5000/api/posts/like/69dae9e0... 5 mins ago
GET localhost:5000/api/posts/following-feed 12 mins ago
PUT thunderclient.com/welcome 14 mins ago
POST localhost:5000/api/comments 20 mins ago
PATCH localhost:5000/api/posts/YOUR_POST... 10 mins ago
POST localhost:5000/api/users/login 12 hours ago
POST localhost:5000/api/posts 9 mins ago
POST localhost:5000/api/users/register just now
PUT http://localhost:5000/api/users/follow/69dafb4245b4e39d3cf62eb8
Send
Query
Headers 3
Auth
Body
Tests
Pre Run
HTTP Headers
Raw
Accept
/
User-Agent
Thunder Client (https://www.thunderclient.
Authorization
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXV
header
value
Status: 200 OK
Size: 35 Bytes
Time: 356 ms
Response
Headers 7
Cookies
Results
Docs
{
"message": "Followed successfully"
}
Response
Chart
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
node - backend
PS C:\Users\Asus\OneDrive\Desktop\IVY COMPROG\final-project-Powerpuffgals-BSIT2a-\backend> node server.js
Local URL: http://localhost:5000
MongoDB Connected Successfully
Incoming Request: PUT /api/posts/like/69dae9e0a3306d8a6744ec81
Auth Middleware: Noe Idanan is verified.
Noe Idanan liked post 69dae9e0a3306d8a6744ec81
Incoming Request: POST /api/users/register
New User Registered: Bubbles Test
Incoming Request: PUT /api/users/follow/69dafb4245b4e39d3cf62eb8
Auth Middleware: Noe Sucero is verified.
Noe Sucero followed Bubbles Test
Finish Setup
Go Live
Search
ENG US
48%
9:57 am
12/04/2026


THUNDER CLIENT
New Request
Activity
Collections
Env
filter activity
DELETE http://localhost:5000/api/posts/69dae9e0a3306d8a6744ec81
Send
Status: 200 OK
Size: 51 Bytes
Time: 281 ms
Response
Headers 7
Cookies
Results
Docs
{
"message": "Post deleted from the ecosystem."
}
Query
Headers 2
Auth 1
Body
Tests
Pre Run
None
Basic
Bearer
OAuth 2
NTLM
AWS
Bearer Token
Token Prefix
Bearer
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Asus\OneDrive\Desktop\IVY COMPROG\final-project-Powerpuffgals-BSIT2a-\backend> node server.js
DELETE request to: /api/posts/69da5911f5632c3ce8f19231
Auth Middleware: Noe Sucero is verified.
DELETE request to: /api/posts/69da5911f5632c3ce8f19231
Auth Middleware: Bubbles Test is verified.
DELETE request to: /api/posts/69da5911f5632c3ce8f19231
Auth Middleware: Bubbles Test is verified.
DELETE request to: /api/posts/69da5911f5632c3ce8f19231
Auth Middleware: Noe Sucero is verified.
DELETE request to: /api/posts/69dae9e0a3306d8a6744ec81
Auth Middleware: Noe Sucero is verified.


THUNDER CLIENT
New Request
Activity
Collections
Env
filter activity
PUT http://localhost:5000/api/posts/like/69dae9e0a3306d8a6744ec81
Send
Status: 200 OK
Size: 31 Bytes
Time: 314 ms
Response
Headers 7
Cookies
Results
Docs
{
"likesCount": 1,
"isLiked": true
}
Query
Headers 3
Auth 1
Body
Tests
Pre Run
None
Basic
Bearer
OAuth 2
NTLM
AWS
Bearer Token
Token Prefix
Bearer
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Asus\OneDrive\Desktop\IVY COMPROG\final-project-Powerpuffgals-BSIT2a-\backend> node server.js
Auth Middleware: Noe Idanan is verified.
PS C:\Users\Asus\OneDrive\Desktop\IVY COMPROG\final-project-Powerpuffgals-BSIT2a-\backend> node server.js
injected env (6) from .env // tip: ⌘ custom filepath { path: '/custom/path/.env' }
Cloud Name Check: dtetztxmo
Server running on port 5000
Local URL: http://localhost:5000
MongoDB Connected Successfully
Incoming Request: PUT /api/posts/like/69dae9e0a3306d8a6744ec81
Auth Middleware: Noe Idanan is verified.
Noe Idanan liked post 69dae9e0a3306d8a6744ec81



VI. GitHub Integration

All backend files were committed and pushed to the GitHub repository inside the /backend folder. We made sure that the commits were clear and properly described the work done during this phase.

We also created a README.md file inside the backend folder. The README.md includes the implemented routes, current models, connection setup, and the steps on how to run the backend locally.

Figure 6. GitHub commit history

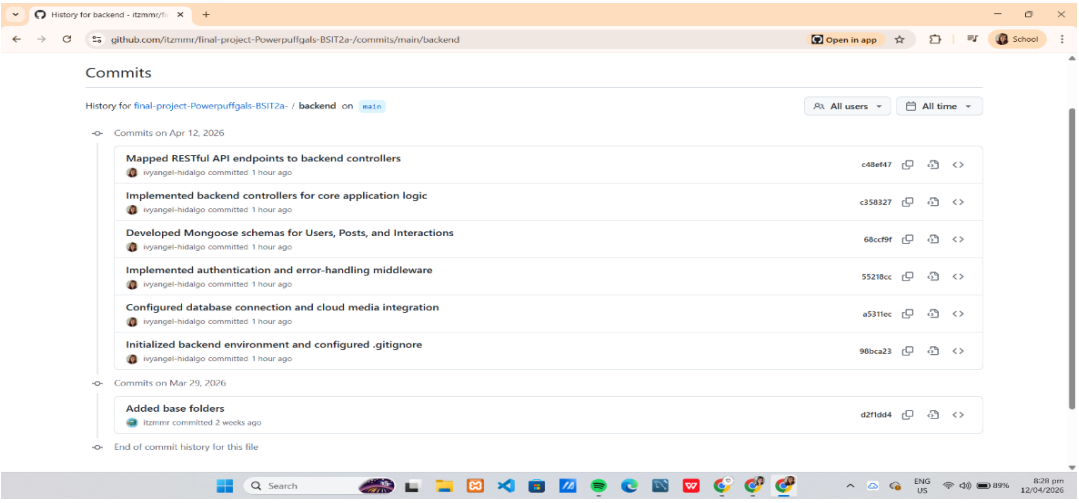


Figure 7. Backend Folder in GitHub repository

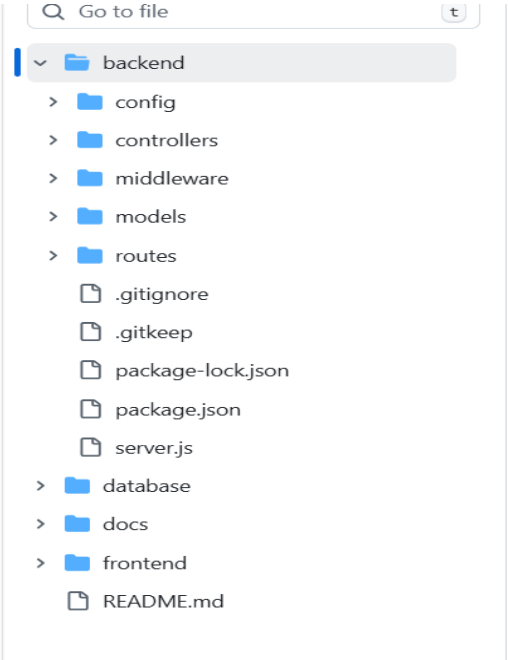
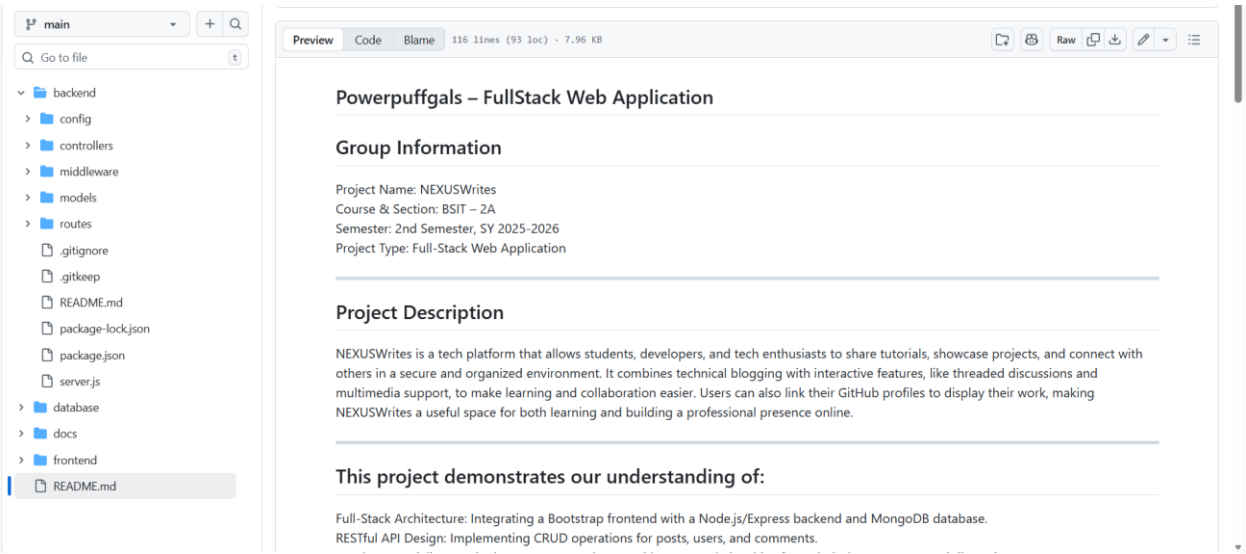


Figure 8. README.md inside the backend folder



VII. Group Member Contributions

Name	Roles	Contribution
1. Ivy Angel R. Hidalgo	Project Manager And Backend Developer	The backend developer builds the server using Express and creates the routes and controllers. She also managed the group and helped in coordinating tasks to ensure the system works properly behind the scenes.
2. Kris Ann B. Apoon	Frontend Developer	She helped the backend developer by testing API connections and making sure the features work smoothly on the user side.
3. April Grace R. Dacanay	Documentation Officer	Takes screenshots and records the progress of the project. Explain how the system works in the laboratory report.
4. Milan R. Rellora	GitHub Manager	Organizes the repository and handles commits. And also making sure all the code is properly pushed with clear messages.
5. Princes Mae L. Mateo	Database Manager	The database manager sets up MongoDB and creates the data models. Our backend developer also helped in organizing the database structure and checking for errors to ensure data is accurate.

VIII. Challenges Encountered

While doing this activity, we encountered some minor challenges such as connecting MongoDB correctly, while making sure the .env file was working, and checking if the API routes were responding properly. There were also small errors in routes path and testing, so we needed to check the code carefully and fix the issues one by one.

Even though we experienced some problems, we were able to solve them by reviewing carefully the code, checking the terminal output, and testing each API routes very carefully. This helped us understand backend development better.

IX. Conclusion

In conclusion, this laboratory activity helped us learn how to build the backend part of a web application using Node.js, Express, MongoDB, and Mongoose. We were able to create a modular backend structure, connect the application to a database, build REST API routes, and test CRUD operations successfully.

This activity is important because it gave us hands on experience in backend development, which is the major part of our final group project . It also helped us understand how the backend and frontend will work together in the next phase. Overall this activity improved our skills in the backend development and API integration.

X. References

- Express Documentation
- Mongoose Guide
- MongoDB Atlas
- Postman
- Thunder Client